

Ejercicio 20: Máxima Sublista No Decreciente (MSND)

Este documento presenta la resolución del problema de encontrar la longitud de la subsecuencia o sublista más larga cuyos elementos se encuentran en orden no decreciente (es decir, creciente no estricto: $a_i \leq a_j$ para todo $i < j$), utilizando la metodología de Programación Dinámica.

1. Ecuación en Recurrencias

Dada una lista $L[1 \dots n]$ de n números enteros, queremos encontrar la longitud de la MSND.

Definición del Estado

Definimos el vector $DP[i]$ como:

$DP[i] =$ La longitud de la Máxima Sublista No Decreciente (MSND) que finaliza obligatoriamente en el elemento $L[i]$ (considerando el prefijo $L[1 \dots i]$).

Donde $1 \leq i \leq n$.

Casos Base

Para cualquier elemento individual por sí solo, la sublista no decreciente mínima que puede formar tiene una longitud de 1 (el propio elemento):

$$DP[i] = 1 \quad \forall i \in \{1, 2, \dots, n\}$$

Relación de Recurrencia

Para calcular $DP[i]$ (con $1 < i \leq n$), buscamos de entre todos los elementos anteriores j ($1 \leq j < i$) aquellos que tengan un valor menor o igual al del elemento actual ($L[j] \leq L[i]$). Esto nos permite "enganchar" el elemento $L[i]$ al final de la secuencia que terminaba en $L[j]$, sumando 1 a su longitud:

$$DP[i] = 1 + \max \left(\{0\} \cup \{DP[j] \mid 1 \leq j < i \wedge L[j] \leq L[i]\} \right)$$

Longitud de la MSND Global

Puesto que la sublista óptima global puede terminar en cualquier posición del vector original, el resultado final de la MSND completa se obtiene buscando el valor máximo de todo el vector DP :

$$\text{Longitud de la MSND} = \max_{1 \leq i \leq n} DP[i]$$

2. Diseño del Algoritmo de Programación Dinámica

A continuación se detalla el pseudocódigo estándar de programación dinámica por acumulación para resolver el problema utilizando índices base 1:

```

Funcion CalcularLongitudMSND(L, n)
Inicio
    // Crear el vector DP de tamaño n + 1 para almacenar los subproblemas
    DP <- Vector de tamaño n + 1

    // 1. Inicialización de Casos Base
    Para i <- 1 Hasta n Con Paso 1 Hacer
        DP[i] <- 1
    FinPara

    // 2. Rellenar el vector DP aplicando la recurrencia
    Para i <- 2 Hasta n Con Paso 1 Hacer
        Para j <- 1 Hasta i - 1 Con Paso 1 Hacer
            // Si el elemento j puede preceder al elemento i en orden no decreciente
            Si L[j] <= L[i] Entonces
                Si DP[j] + 1 > DP[i] Entonces
                    DP[i] <- DP[j] + 1
            FinSi
        FinSi
    FinPara

    // 3. Buscar la longitud máxima alcanzada en cualquier posición final
    longitud_maxima <- 0
    Para i <- 1 Hasta n Con Paso 1 Hacer
        Si DP[i] > longitud_maxima Entonces
            longitud_maxima <- DP[i]
        FinSi
    FinPara

    Retornar longitud_maxima
FinFuncion

```

3. Análisis de Eficiencia

- **Complejidad Temporal:** $\Theta(n^2)$. El algoritmo utiliza dos bucles anidados para calcular la matriz unidimensional de estados. El bucle externo recorre los n elementos y el interno examina para cada paso i todas las posiciones anteriores $j \in [1, i - 1]$ para comprobar la condición y buscar el máximo. Las operaciones interiores son comparaciones constantes $O(1)$. El número de operaciones de comparación es:

$$\sum_{i=2}^n (i - 1) = \frac{n(n - 1)}{2} \in \Theta(n^2)$$

- **Complejidad Espacial:** $\Theta(n)$ para almacenar los n estados del vector DP .

4. Propuesta de una Solución Eficiente de Coste $O(n \log n)$

La solución anterior de coste cuadrático $\Theta(n^2)$ se puede optimizar drásticamente utilizando una estrategia de búsqueda binaria sobre un vector auxiliar de candidatos activos. Este enfoque se conoce clásicamente como el **Algoritmo de la Paciencia** (*Patience Sorting*).

Idea del Algoritmo Óptimo

1. Mantenemos un vector dinámico auxiliar T donde $T[k]$ almacena el **menor valor final** de todas las sublistas no decrecientes de longitud k encontradas hasta el momento.
2. Este vector T estará garantizadamente **ordenado de forma estrictamente creciente**.
3. Recorremos los elementos $L[i]$ de la lista de izquierda a derecha de uno en uno:
 - Utilizamos **búsqueda binaria** en el vector T para encontrar el primer elemento que sea estrictamente mayor que $L[i]$. Sea p la posición de dicho elemento en T .
 - Si todos los elementos de T son menores o iguales a $L[i]$, significa que podemos extender la MSND más larga que tenemos. Añadimos $L[i]$ al final de T .
 - Si encontramos una posición válida p , actualizamos el valor: $T[p] \leftarrow L[i]$ (un final más pequeño y, por tanto, más prometedor para futuras extensiones de longitud p).
4. Al terminar de procesar toda la lista, la longitud del vector auxiliar T será exactamente la longitud de la MSND global.

Pseudocódigo de Coste Logarítmico

```

Funcion CalcularLongitudMSND_Eficiente(L, n)
Inicio
    // T[k] guardará el menor valor con el que puede terminar una MSND de longitud k
    T <- Vector dinámico inicialmente vacío

    Para i <- 1 Hasta n Con Paso 1 Hacer
        elemento_actual <- L[i]

        // Buscamos mediante búsqueda binaria el primer elemento en T que sea estrictamente mayor que 'elemento_actual'. Al ser no decreciente (<=), buscamos la cota superior
        p <- BuscarPrimerMayor(T, elemento_actual)

        Si p = -1 Entonces
            // Si no hay ningún elemento mayor, el elemento actual extiende la subsecuencia
            Añadir elemento_actual al final de T
        Sino
            // Si existe, reemplazamos el valor para optimizar el final de esa subsecuencia
            T[p] <- elemento_actual
        FinSi
    FinPara

    // La longitud del vector T es la longitud de la MSND
    Retornar Tamaño(T)
FinFuncion

Funcion BuscarPrimerMayor(T, valor)
Inicio
    izq <- 1
    der <- Tamaño(T)
    resultado <- -1

```

```
Mientras izq <= der Hacer
    mid <- ParteEntera((izq + der) / 2)

    Si T[mid] > valor Entonces
        resultado <- mid
        der <- mid - 1 // Buscar más a la izquierda para encontrar el primero
    Sino
        izq <- mid + 1 // Buscar a la derecha
    FinSi
FinMientras

Retornar resultado
FinFuncion
```

Análisis de Coste de la Solución Eficiente

- **Complejidad Temporal:** $O(n \log n)$. Realizamos un único bucle que recorre secuencialmente los n elementos de la lista de entrada. Para cada elemento, la localización de su punto de inserción/reemplazo en el vector ordenado T se realiza mediante una búsqueda binaria que tiene un coste garantizado de $O(\log n)$. Por lo tanto, el coste total es estrictamente logarítmico.
- **Complejidad Espacial:** $O(n)$ en el peor caso para almacenar el vector auxiliar T (si la lista estuviera ya ordenada en orden no decreciente de origen, T guardaría los n elementos).